

Display · adopted unit folders, shown and accepted on the page

1 The same folder shape as a paper, at a new address

The plugin's full tree: one display/ per page, one folder per unit, every unit the same shape.

```
<page>/
  <page>.md          the page, on stage
  display/          the plugin . discovery never enters
    <PageID>-Display1-<slug>/    ONE rendered unit
      README.md      claim . kind . caption-job . fragility . status
      intake/        caller-owned, provenance-bound values
        manifest.yaml
        inputs/      small approved summary CSV or JSON extracts only
      recipe/        renderer-owned script, spec, receipts
      float.tex      caption + label + asset reference
      preview.tex    standalone wrapper that inputs float.tex
      preview.pdf    compiled look, what the tab frames
      assets/        the WINNING render
      candidates/    pre-decision renders
      versions/      superseded variants, kept for history
    <PageID>-Display2-<slug>/    the next unit, same shape
```

A page keeps its figures in the same folder shape a paper uses, so nobody has to learn a second one. The tree above is copied from the display family's own rules file, `display-unit-output-contract.md`, word for word. The folders split in two. `intake/` and `recipe/` are yours to write, filled by people and by drawing skills. `assets/` and `preview.pdf` are rebuilt for you from those two. So a rebuild may replace the built half, and it never touches the half you wrote. The one page-side change is the address. The unit sits inside the page's own `display/` folder, so when the page moves, its figures move with it.

2 Every unit shows as a card, and a sentence can point at it

The tab: what a reader sees when they open Display on a page.

```
right pane . Display
+-----+
| QPf5-Display1-pipeline-tikz          |
|   preview.pdf, framed (body.py ccpdf) |
|   README status . rebuild            |
|   accepted: [ ] . shown, NOT tickable |
+-----+
one card per unit . empty display/ = empty state, not an error
```

Each unit shows as one card in the right pane, and naming its id in a sentence turns that id into a button. The tab is built the way the Slides tab is. A small button in the page's toolbar asks the server for this page's units. The server then writes a view that frames each unit's `preview.pdf`.

builds `preview.pdf` again from `preview.tex`. It never reruns the recipe, never changes the inputs, and never changes a yes. A unit is EVIDENCE. Name a unit's id in plain text, and that id turns into a small button in the sentence. Click it, and the unit's card opens with its picture and its yes-or-no state. There is a second way to attach the same evidence, made for machines. It is a `> Display:` line under a sentence, beside the `> Citation:` and `> Value:` lines. That way a program can add evidence without ever editing a person's words. You may point at a unit that has no picture yet, in either form. The button then says a drawing is still owed. The exports carry the evidence too. The LaTeX PDF prints a named unit as a real figure, right after the paragraph that names it. The Word file puts the picture in with its caption, and adds a comment note on the sentence.

3 Three hands touch a unit, and only one may say yes

Who writes what: three hands touch one unit, and each stays in its own half.

```

renderer skills    haipipe-display-table . -figure . -diagram . -illustration
                  write recipe/ . assets/ . preview.pdf
a person          rules intake/ and ticks accepted:
the pane          shows everything . writes NOTHING

```

Drawing skills fill the folder, but the yes on a picture is always a person's click. The drawing skills already belong to the display family, and this page adds none of its own. The `accepted:` row is the one rule with teeth. A `[x]` there means a person looked at one exact picture and said yes. A machine that ticks it has faked a judgment, not saved time, so no machine may tick it.

The kinds, and who draws them: one door sends the ask to five drawing skills, four scripted and one written by hand.

```

haipipe-display . the DOOR . say the ask, it picks the renderer
data-driven . needs a ruled intake/ of numbers
  -table      aggregated CSV/JSON > booktabs LaTeX . recipe = the spec
  -figure     results > plot . recipe = the python script
concept-driven . the intake is a spec, not numbers
  -diagram    FigureSpec JSON > deterministic editable SVG
  -tex        .tex recipe > vector PDF . TeX-native . a person writes it
  -illustration image-gen prompt > concept figure . recipe = prompt + receipts

```

A data-driven skill draws numbers, and it may not go back to the raw data. It reads only the small approved copy in `intake/`. A concept-driven skill draws an idea, so it holds no numbers at all, and its input is the sketch itself. In the row a person draws by hand: they type the picture as `.tex` code, and that is how both units on this page were drawn. It gets its own row because the drawing is TeX code, so it uses the document's own fonts and math, and `float.tex` can pull the file in as it is. The skill is named after that way of drawing, not after one package, so `haipipe-display-tex` covers a TikZ figure, an algorithm block, and a math display (JL 260816). Such a unit still owes a built `assets/figure.pdf`. A document that pulls the code in rarely has the same setup lines at the top of the file. Drawing a unit again from its recipe depends on the kind. A python figure reruns its script, a diagram redraws from its spec, and an AI picture replays its saved prompt.

4 The order of the work, and which steps need a person

The five steps: from approved numbers to a picture a person accepts, and who moves each step.

- (1) INTAKE the caller fills intake/: manifest.yaml + small
 approved extracts, every value provenance-bound
- (2) RENDER the matching renderer skill runs, dispatched through
 task-for-display; it fills recipe/ + candidates/
- (3) PICK a person picks the winner; it moves to assets/,
 the losers stay in candidates/ or versions/
- (4) BUILD preview.tex compiles to preview.pdf; the adapter's
 --check reports a stale unit instead of hiding it
- (5) ACCEPT a person reads the render and ticks accepted:

Nothing is drawn before a person approves the numbers, and nothing is finished before a person looks. Drawing never comes first. With no approved inputs there is nothing safe to draw, and the intake rules exist to refuse exactly that. Steps (1) (3) (5) belong to a person, and steps (2) (4) belong to the machine. Figure 1 draws that split of hands. The tab's reruns step (4)'s build alone, never step (2)'s drawing skill. If the inputs change, the work runs forward again from (2) to (5). The old yes drops back to [], because the picture a person agreed to no longer exists.

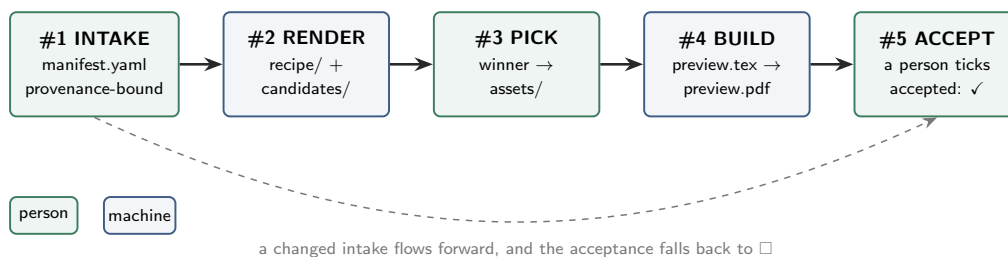


Figure 1: The display pipeline: five steps from approved intake to an accepted render. Green steps belong to a person, blue steps to machinery, and a changed intake returns the acceptance to open.

Where step (1)'s numbers come from: each hop points at the one before it by id, never by a copy.

```

the page needs a number    > raised as a PROBE . the page's own probe/ plugin (QPf9)
probe orchestrator         > sweeps the bank . reuse | enrich | fresh
haipipe-task runs          > answer lands in <task-folder>/QA/<n>-<slug>.md
                           + the run's artifact, e.g. counts.csv . CANONICAL
intake/manifest.yaml       > cites that holder BY ID . carries only a
                           small approved extract in inputs/
    
```

A probe is a question card. The page asks its question once, the answer lands in the shared task list, and after that everyone points at the card's id instead of asking again. So `intake/` is not a second copy of the data. The task's own output stays the one source, and the manifest only points at it, by id. A unit names its evidence by id, never by a file path, and it never types a number by hand. Figure 2 draws that move: ask once, point at it twice. The slides plugin already works

this way on QPf3. Its deck asks for an id, a small helper turns the id into the file’s real path, and the template holds no digits at all. So a hand-typed number has nowhere to live. That id lookup already reaches units under a page, because the citation index finds a unit by id wherever the unit sits. Aim A1 asks for the other half. The build adapter should accept a page’s `display/` as a unit root, so `--check` can report an out-of-date page unit instead of hiding it.

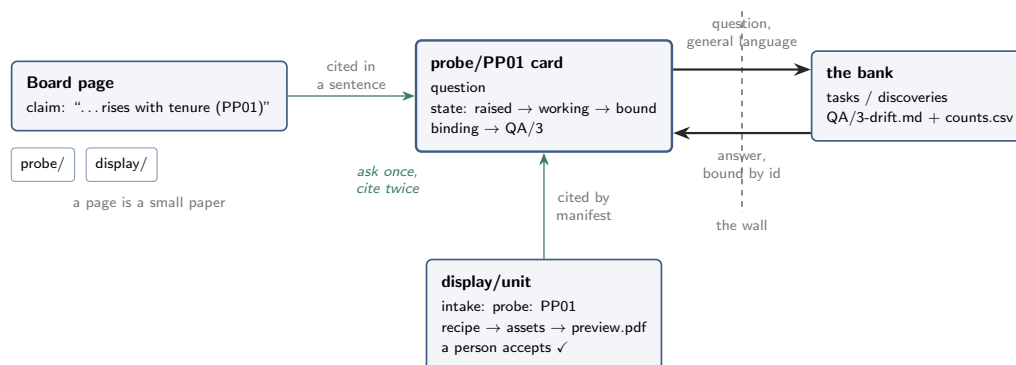


Figure 2: A page as a small paper: the probe plugin asks a question once, the bank answers behind the wall, and both the display intake and the page’s own sentence cite the same card by id.